C Library

SESSION OBJECTIVE

- Understanding functions in C, in detail.

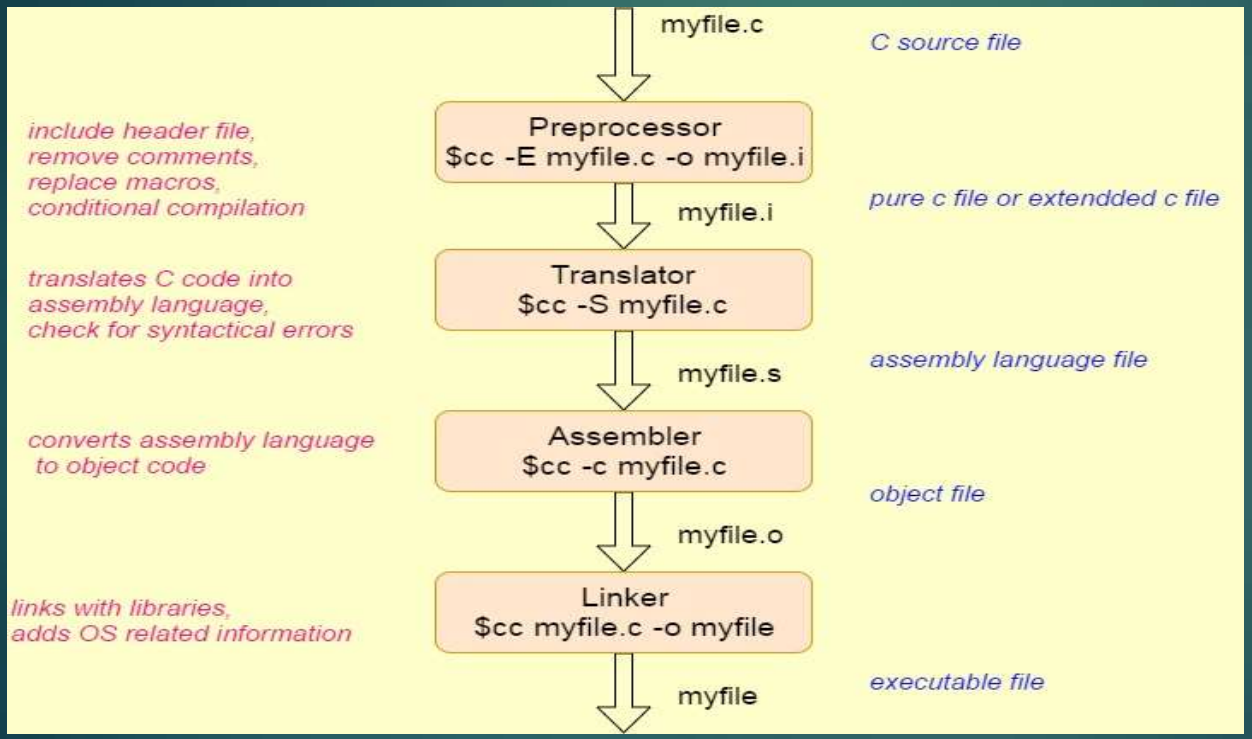
- Two categories of functions
  - Library functions
  - User defined functions
- Two types of Libraries
  - Static library
  - Dynamic library
- Three types of Linkage
  - Static
  - Loadtime
  - Runtime

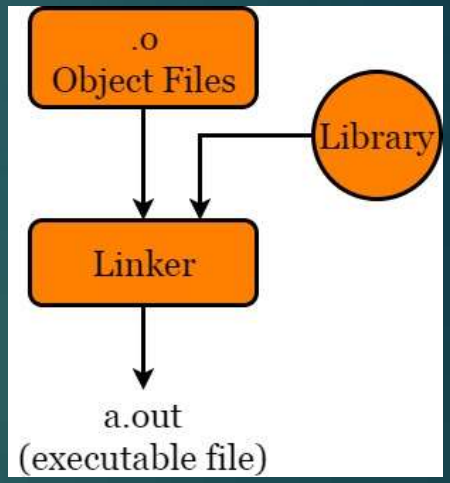
Library basics

- When a program is written in C language, to execute the written program, it has to go through compilation stages.
- When a C program is compiled, the compiler generates object code.
- After generating the object code, the compiler also invokes linker.
- One of the main tasks for **linker** is, to make code of library functions available to your program.
- **Library is a binary file**(compiled), having collection of functions. For example printf, scanf, strcpy, pow, fopen, malloc etc..
- Libraries are typically names with the prefix "lib". This is true for all the C standard libraries.

Linker

- A linker can accomplish the task of linking in two ways
  - by copying the code of library function to your object code
  - by making some arrangements so that the code of functions is not copied, but made available at run-time.





Static library

- Static Libraries is the result of the linker making copy of all used library function definitions into the executable file.
- Functions from static library have static linking.
- Static Linking creates larger size executable files, hence require more memory.
- Another name for static library is “archive file”. e.g, libc.a (all C function definitions), libm.a (all mathematics related function definitions), libvector.a
- Examples of static libraries (libraries which are statically linked) are, .a files in Linux and .lib files in Windows.

Static library

- Since this library code is connected at compile time, the final executable has no dependencies on the library at run time i.e. no additional run-time loading costs, it means that you don't need to carry along a copy of the library that is being used and you have everything under your control and there is no dependency.

- In static libraries, once everything is bundled into your application, you don't have to worry that the client will have the right library (and version) available on their system.

- One major advantage of static libraries being preferred even now "is speed". There will be no dynamic querying of symbols in static libraries. Many production line software use static libraries even today.

- One drawback of static libraries is, for any change (upgradation) in the static libraries, you have to recompile the main program every time.

Dynamic Library

- Dynamic libraries help in dynamic linking, i.e. dynamic load time linking and dynamic run time linking.
- Another name for dynamic library is “shared object” or “DLL files” in windows. So the extension is .so in linux and .dll (Dynamically Linked Library) in windows. For eg, libc.so, libm.so , libvector.so, libxyz.dll
- When linker is dynamically linking with dynamic libraries, required function definitions are not copied to executable file. Thus executable file size is less.
- When you link an application against a shared library, the linker leaves some stubs (unresolved symbols) to be filled at application loading time. These stubs need to be filled by a tool called, dynamic linker at run time or at application loading time.
- Loading of a library is of two types, static loading and dynamic loading.

Static Loading

- In static loading, all of those dependent shared libraries are loaded into memory even before the application starts execution.
- A loader examines application's dependency on shared libraries.
- If these libraries are already loaded into the memory, the library address space is mapped to application virtual address space (VAS) and the dynamic linker does relocation of unresolved symbols.
- If these libraries are not loaded into memory the loader searches in standard library paths and loads them into memory, then maps and resolves symbols.

Dynamic Loading

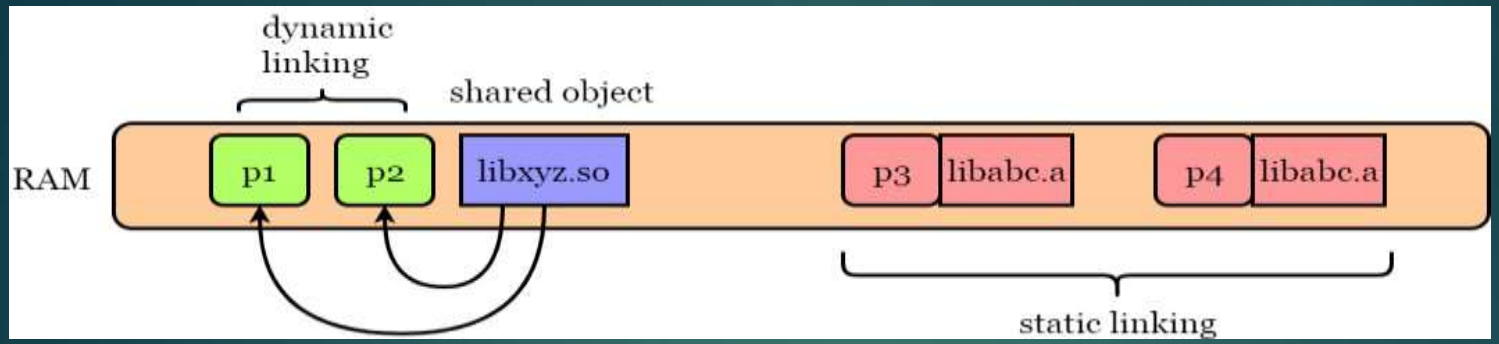
- Dynamic loading is about loading of library on demand, You can invoke loading of these shared libraries dynamically when you need their functionality.
- In this case, the programmer aware of situation, when should the library be loaded. The tool-set and relevant kernel provides API to support dynamic loading, and querying of symbols in the shared library.
- size of executable programs is dramatically reduced because each program does not have to store redundant copies of the library
- Dynamically linked applications are dependent on libraries. If library is not present, applications won't work. Such applications may face version mismatch issue.

- Look at the program in next page, we will learn the linkage of sum, mul, printf with main .
- Revise the following commands and tools
  - gcc –c
  - nm
  - size
  - file
  - ldd
  - ar

```
#include<stdio.h>
#include<stdlib.h>
int mul(int,int);
int sum(int,int);
int main()
{       int x=10,y=20,result,choice;
        printf("enter 1:sum 2:mul");
        scanf("%d",&choice);
        if(choice==1)                   result=sum(x,y);
        else if(choice==2)              result=mul(x,y);
        else       {            printf("ERROR: Invalid choice\n");  return 0;        }

         printf("result=%d\n",result);

}
```



- In dynamic linking, object files are not combined with programs at compile time, also they are not copied permanently into the final executable file
- therefore, a shared library reduces the size of final executable.
- Shared libraries or dynamic link libraries (dlls) serve a great advantage of sharing a single copy of library among multiple programs(processes), hence they are called shared libraries, and the process of linking them with multiple programs is called dynamic linking.

- Here main() is a calling function and sum is a called function.
- When main() knows where exactly the sum() function is present?
- Where: after address resolution
- When:
  - 1) compile time
  - 2) load time
  - 3) run time

- 1) Compile Time: at compile time only, main knows where sum is located then this linking is called as static linking or static binding.
- 2) Load Time: at load time main knows the resolved address for sum function. This is known as dynamic load time linking. Better to say, it is dynamic linking with static loading.
- 3) Run Time: here main function knows the sum function's resolved address at the program execution time. This called as dynamic run time linking or dynamic linking with dynamic loading.
- The static library is used only for static linking but shared objects are used in both dynamic load time and dynamic run time linking.

Dynamic Runtime linking

- Dynamically run time linking is also known as dynamic linking with dynamic loading. These are helpful in implementing plugins because they permit waiting to load the plugin until it's needed.
- In this kind, when a particular function is needed from xyz library then only it is loaded into RAM and once work is done, library is unloaded whereas program is still running in RAM. They're also useful for implementing interpreters that wish to occasionally compile their code into machine code and use the compiled version for efficiency purposes, all without stopping. For example, this approach can be useful in implementing a just-in-time compiler.
- In Linux, DL libraries aren't actually special from the point-of-view of their format; they are built as standard object files or standard shared libraries. The main difference is that the libraries aren't automatically loaded at program link time or start-up; instead, there is an API for opening a library, looking up symbols, handling errors, and closing the library. C users will need to include the header file <dlfcn.h> to use this API.